

4.2 Vision Transformers

Non-overlapping patches, a linear map, positions, no causal mask.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A **vision transformer (ViT)** is the Week 3 block on **patches** instead of words. Once an image is a sequence, every LLM trick (attention, positions, [CLS], later CLIP) applies. The recipe is patchify, linear map, positions, **no causal mask**.

First time this method appears. A **ViT** treats an image as a sequence of patches.

What. Cut the image into non-overlapping patches, linearly map each to a d -vector, add positions, run a transformer **without** a causal mask. **Why.** CNNs are the older default. ViT shows the same block as language can mix patches globally from layer 1. **Architecture.** Patch embed $E \in \mathbb{R}^{P \times d}$ plus optional CLS token, then encoder blocks. Classification: CLS or mean pool, then a head. **How.** Lab 4 flattens patches. No convolution is required for the cartoon. Pretrain on a large image set; small data prefers CNNs or a strong recipe. **Formula.** For patch size $p \times p$ and image $H \times W$, sequence length $T = (H / p) \cdot (W / p)$. $X = \text{LN}(PW_e + E_{\text{pos}})$. **Tradeoffs.** + Global mixing, same code as NLP. – Quadratic in patches; needs data or regularization; patch size is a method knob.

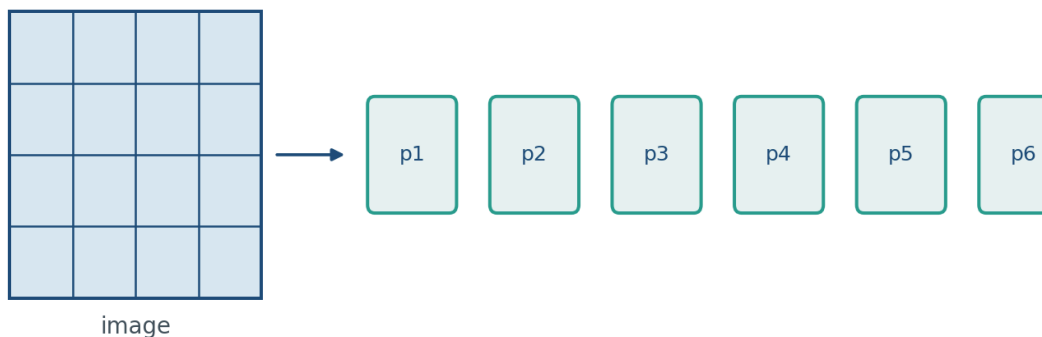
1. Patches as tokens

Split a $H \times W$ image into non-overlapping $P \times P$ patches, flatten each patch, map with a linear layer to width d .

$$N = HW / P^2.$$

Add a learned **[CLS]** if you classify from one vector, and add **positions** (a 1-D index, or row–column). Attention without positions is a bag of tiles.

A ViT treats patches the way an LLM treats tokens



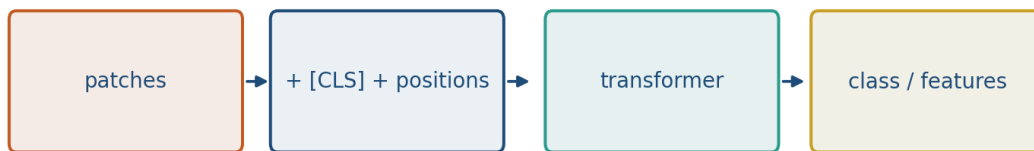
A CNN shares a small kernel and builds a hierarchy. A ViT sees global context in layer 1, at $O(N^2)$ cost. For a 224 image and $P = 16$, $N = 196$, which is a short paragraph.

Flattening a $16 \times 16 \times 3$ patch is 768 numbers. That linear map is the same as a convolution with kernel P , stride P , d output channels. RGB is just three extra channels in the flatten.

Do not count overlapping CNN windows and then write $N = HW / P^2$. ViT patches **tile**.

2. The same stack, no language mask

Vision transformer: same block as language, different tokens



Dosovitskiy et al. (2021): ViTs need more data than ResNets if trained from scratch; they shine with scale. In this course you will almost always **start from a pretrained** ViT (CLIP's image tower, or a Hugging Face checkpoint), not from random pixels.

Unlike GPT, a classifier ViT is **bidirectional**: every patch may look at every patch. There is no future to hide. Pooling: [CLS] as in BERT, or **mean-pool** the patch tokens then a linear head (also common).

Hybrid: a small CNN stem, then a transformer on a coarser grid. Same N idea, fewer tokens.

CLIP's image tower is often this ViT. When you later freeze "the vision encoder," you are freezing patch embed + transformer. You are not freezing a ResNet unless the checkpoint says so.

3. Teaching this note

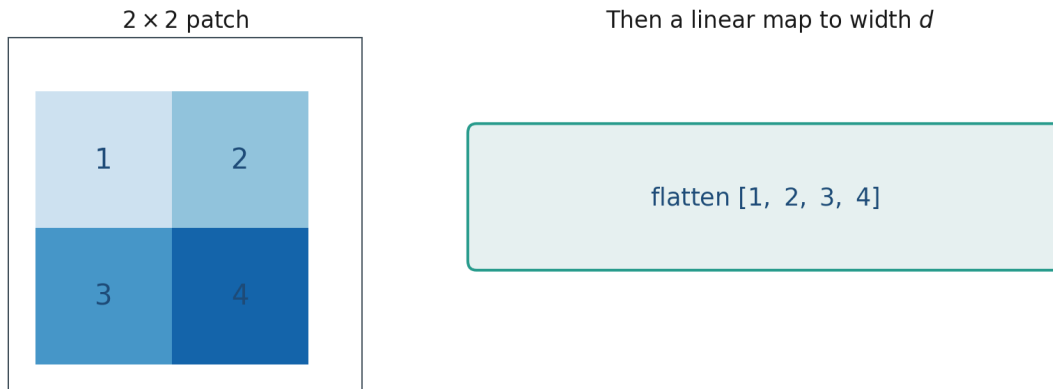
~16 minutes. Count patches on a square image, flatten one toy 2×2 patch, add [CLS]+positions, then "same block as Week 3, no causal mask." Play the **patch embedding** stretch of the Umar Jamil video. Lab 4 will flatten patches in numpy; do one flatten on the board first. Do not train a ViT in this block.

4. Worked example

Image $H = W = 4$, $P = 2$, 1 channel. Then $N = 4$ patches. Top-left patch

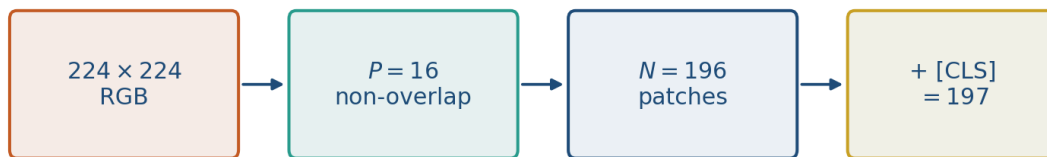
$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

flattens to $\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$. A linear map $W \in \mathbb{R}^{2 \times 4}$ with first row all 0.25 sends it to a 2-D token whose first coordinate is the mean 2.5.



Standard ImageNet: 224×224 , $P = 16$, $N = 196$. Plus [CLS] : **197** tokens. Attention map $197^2 = 38809$ scores per head.

A 224 image is a short paragraph. No causal mask: every patch may look.



A smaller patch ($P = 8$ on 224) gives $N = 784$ tokens and a heavier N^2 map. That is why 16 is the default: enough spatial pieces, still a short sequence.

You do not implement ViT from random init in this course. You count patches, you add positions, you reuse Week 3.

5. Where students get stuck

- Using overlapping CNN-style windows and then miscounting $N = HW / P^2$.
- Forgetting positions so the model cannot tell top from bottom.
- Putting a GPT causal mask on patches (there is no "next patch" to hide).
- Training a ViT from scratch on a tiny medical set.

6. Video

Watch [Umar Jamil: Vision Transformer / VLM walkthrough](#).

Pause on **image patches as tokens**, the linear patch embedding, and positional encodings. That is the ViT; CLIP/BLIP wrap extra towers around it.

7. Practice

1. Count tokens for a 384×384 image, patch 16.
2. Why add positions if attention already sees all patches?
3. A medical image dataset with 800 scans: from-scratch ViT or a pretrained tower? Why?
4. For 224×224 , $P = 16$, include a [CLS] token. How many tokens enter the transformer? How many scores in one attention map?
5. A 32×32 RGB image, $P = 8$. Flatten one patch: how many numbers before the linear map? How many patches in the whole image?
6. Why does a classifier ViT **not** use a causal mask?
7. The patch linear map is a convolution with which kernel size and stride?